Dhs5 Utility

Short reference for the main systems: one page per system.

01	Custom Settings	Singleton settings in Project Settings / Preferences, PlayerPrefs-backed values	2
02	Database	UID-indexed asset containers with a dedicated editor window	3
03	Debugger	Category logger, console window, on-screen console & commands, runtime inspector	4
04	Updater	Update channels, delayed calls, timelines, update conditions	5
05	Save & Load	Category-based JSON save files and ordered coroutine loading	6
06	UI	uGUI controls with C# events, stackable transitions, navigation boxes	7
07	Scene Management	`SceneReference` field and the Scenes browser window	8
08	Player Loop	Shared, prioritised and restorable PlayerLoop modification	9
09	Ticket System	Lightweight unique handles for cancellable registrations	10
10	Attributes	Inspector attributes: ReadOnly, Show, HelpBox, pickers, Creator...	11
11	Custom Properties	Serializable wrappers: EnabledValue, EnumValues, dates, types	12
12	General Utility	Extension methods, editor asset & GUI helpers, reflection, script generation	13

COMMON CONVENTIONS

- Every editor window lives under **Window > Dhs5 Utility**.
- Systems configured from an asset (Settings, Databases, Debugger, Updater, Save) load it from a `Resources/<System>` folder. The asset is created from the system's window ("Create new asset" or Refresh).
- Enum-driven systems (debug categories, update channels and conditions, save categories) **generate a C# enum** from the asset. After any change, press **UPDATE ... SCRIPT**. Deleting or reordering entries shifts serialized enum values.
- **ENSURE ASSET SANITY** (Settings tab of each window) repairs a system's asset list and folder.

01 Custom Settings

`Dhs5.Utility.Settings`

Singleton settings `ScriptableObject`s, reached from anywhere with `MySettings.I`. Each one gets a page in **Project Settings** or **Preferences** and in the Settings window. They can hold **PlayerPrefs-backed values** (a default set in the editor, the player's value persisted at runtime) and inline **sub settings** assets.

KEY TYPES

`CustomSettings<T>`	Base class to inherit (`where T : CustomSettings<T>`). `static T I` lazily loads the asset.
`[Settings(path, Scope)]`	Required class attribute. `Scope.Project` → Project Settings, asset in `Assets/Resources/Settings/`, ships in builds. `Scope.User` → Preferences, asset in `UserSettings/`, editor only.
`PlayerPrefMember<T>`	Serialized key + default. `Value` get/set reads and writes PlayerPrefs in play mode; implicit conversion to `T`.
`PlayerPref…`	`Int, Float, Bool, String, Vector2, Vector2Int, Vector3, Vector3Int`. Vectors use one key per component.
`[SubSettings]`	On a ScriptableObject field: the asset is drawn inline. Created next to the settings as `SUBS_<Field>`.
`ISubSettingsEditor`	Implement on a custom Editor to control how a sub settings asset is drawn inline.
`BaseSettingsEditor`	Inspector for all settings. Override `OnTitleBarGUI` / `OnFooterBarGUI` to customise the page.

SETUP

1. Declare `class MySettings : CustomSettings<MySettings>` with `[Settings("My/Path", Scope.Project)]`.
2. Open **Window** > **Dhs5 Utility** > **Settings** and press **Refresh**, or press **Create** on its Project Settings page. Assets are only created this way.
3. Add serialized fields as usual. Use `PlayerPrefXxx` fields for player-persisted values; keys default to `<Type>.<propertyPath>`.
4. A non-abstract subclass of a settings class replaces it, at the same path unless the subclass has its own `[Settings]`.

EXAMPLE

```
using Dhs5.Utility.Settings;
using UnityEngine;

[Settings("Game/Audio", Scope.Project)]
public class AudioSettings : CustomSettings<AudioSettings>
{
    [SerializeField] private float m_fadeDuration = 0.5f;
    [SerializeField] private PlayerPrefFloat m_masterVolume = new();
    [SerializeField] private PlayerPrefBool m_muted = new();

    public static float FadeDuration => I.m_fadeDuration;
    public static PlayerPrefFloat MasterVolume => I.m_masterVolume;
    public static PlayerPrefBool Muted => I.m_muted;
}

// Anywhere at runtime
float vol = AudioSettings.MasterVolume.Value;   // PlayerPrefs value, or the default
AudioSettings.MasterVolume.Value = 0.8f;        // saved to PlayerPrefs immediately
bool muted = AudioSettings.Muted;               // implicit conversion
```

EDITOR TOOLING

- **Window/Dhs5 Utility/Settings**: a dropdown of every settings type by path, a sub-settings references mode (create, place in folder, delete, ensure validity) and Refresh.
- The PlayerPref drawer edits the default value and the key. In play mode it shows the live value read-only.

GOOD TO KNOW

- Outside play mode, `Value` returns the default. In play mode, the PlayerPrefs value is loaded once per session; this works with domain reload disabled.
- Declare PlayerPref fields with `= new()`. Changing a key in the inspector deletes the old PlayerPrefs key.
- User-scope settings are editor only (`I` is null in builds) and do not support `[SubSettings]`.
- When overriding `OnValidate`, call `base.OnValidate()`.

02 Database

`Dhs5.Utility.Databases`

A database is a **Container** asset: a list of asset references (ScriptableObjects, or components on prefabs) that implement `IContainerElement`. Each element has a stable **int UID** and an optional **alias**. Databases are singletons found by type in `Resources/Databases` and are edited in a window with a searchable, column-based list.

KEY TYPES

`Container`	Base ScriptableObject. `Count`, `GetByUID<T>` / `TryGetByUID`, `GetAtIndex`, `GetObjects()`, `GetObjectsOfType<T>()`, `TryFindObjectByAlias<T>` (case-insensitive).
`[Database(path, types)]`	Turns a Container into a database. `path` is its entry in the window; `types` are the allowed element types (subtypes included).
`Database (static)`	`Get<T>()`, `Enumerate<T,U>()`, `GetDataAtIndex<T,U>()`, `GetDataByUID<T,U>(uid, out U)`, `ClearInstances()`.
`IContainerElement`	`int UID`, `string Alias`, and an editor-only `Editor_SetUID(int)`.
`[ContainerDisplay]`	On an element field: shows the field as a column (`width`, `priority`).
`ContainerEditor`	The list inspector. Subclass it, or override the Container's `Editor_*` hooks, to customise it.

SETUP

1. Write the element: a ScriptableObject (or prefab MonoBehaviour) implementing `IContainerElement` with a serialized UID.
2. Write the database: `[Database("Game/Items", typeof(Item))] class ItemDatabase : Container {}`.
3. Open **Window** > **Dhs5 Utility** > **Databases** and press **Refresh**. This creates `Assets/Resources/Databases/ItemDatabase.asset`.
4. Add elements with **+** (find an existing asset or create a new one of an allowed type) or by drag and drop. The UID is computed from the asset GUID; collisions get an offset.

EXAMPLE

```
public class Item : ScriptableObject, IContainerElement
{
    [SerializeField] private int m_uid;
    [SerializeField, ContainerDisplay(width = 120f, priority = 1)] private string m_alias;
    [SerializeField, ContainerDisplay(width = 60f, priority = 2)] private int m_price;
    public int UID => m_uid;
    public string Alias => m_alias;
    public int Price => m_price;
#if UNITY_EDITOR
    public void Editor_SetUID(int uid) => m_uid = uid;
#endif
}

[Database("Game/Items", typeof(Item))]
public class ItemDatabase : Container { }

// Runtime
var db = Database.Get<ItemDatabase>();
if (db.TryFindObjectByAlias<Item>("sword", out var sword)) Debug.Log(sword.Price);
foreach (var item in Database.Enumerate<ItemDatabase, Item>()) Debug.Log(item.name);
Database.GetDataByUID<ItemDatabase, Item>(savedUid, out var fromSave);
```

EDITOR TOOLING

- **Window/Dhs5 Utility/Databases**: a path dropdown, a Debug toggle (raw inspector) and Refresh.
- Container list: search, resizable UID / Name / `[ContainerDisplay]` columns, and the focused element's inspector below. Right-click an entry for Ping, Open Asset, Remove or Delete Asset; **Delete** removes it; double-click opens it.

GOOD TO KNOW

- `Database.Get<T>()` only looks in `Resources/Databases`. It returns null if the asset was never created.
- UIDs are only assigned when an element is added through the editor UI. Elements with UID 0 are flagged invalid; the editor offers to fix them.
- Duplicate UIDs log an error and only the first element is kept in the lookup.
- `Dhs5.Utility.OldDatabase` has its own `DatabaseAttribute`. Don't import both namespaces in the same file.

03 Debugger

A **category-based logger**. Each category has a colour and a verbosity level, and log lines are prefixed with the coloured category name. It also provides an editor **log console** with per-category filters, an **on-screen log and command console** (editor and development builds), and a **Runtime** tab that inspects `[RuntimeDebug]` members live.

KEY TYPES

`Logger`	`Log(category, msg, level = 2, onScreen = false, context = null)`, `LogWarning`, `LogError`, `LogAlways`, `LogOnScreen(category, msg, type, level, duration)`.
`EDebugCategory`	Enum generated from the categories (plus `EDebugCategoryFlags` and `HasCategory()`).
`DebuggerAsset`	`Resources/Debugger/Debugger.asset`: categories (`DC_<NAME>` assets), console settings, max logs and on-screen sizes.
`[ConsoleCommand(name, scope)]`	On a **static** method: makes it a console command. Scope is `RUNTIME` (default), `EDITOR` or `BOTH`.
`[RuntimeDebug]`	On fields, properties or parameterless void methods, which appear in the Runtime tab. Register objects with `RuntimeDebugger.Register(bool, category, obj)`.
`OnScreenConsole`	Created automatically if enabled: `Open()`, `IsOpened`, events `Opened` / `Closed`.

SETUP

1. Open **Window** > **Dhs5 Utility** > **Debugger**. In the **Settings** tab, click **Create new asset**.
2. In **Categories**, click **ADD NEW CATEGORY** (BASE is always first; up to 32), then set the name, colour and level (-1 = off, 0–2). Press **UPDATE CATEGORY SCRIPT**.
3. On-screen console: in Settings, tick *Enable On Screen Console* and assign the open/close `InputActionReference`s (Input System).

EXAMPLE

```
public class Player : MonoBehaviour
{
    [RuntimeDebug] private int m_health = 100;
    [RuntimeDebug] public bool IsDead => m_health <= 0;

    private void OnEnable()  => RuntimeDebugger.Register(true, EDebugCategory.GAME, this);
    private void OnDisable() => RuntimeDebugger.Register(false, EDebugCategory.GAME, this);

    public void TakeDamage(int dmg)
    {
        m_health -= dmg;
        Logger.Log(EDebugCategory.GAME, "Took " + dmg, level: 1, context: this);
        if (IsDead) Logger.LogWarning(EDebugCategory.GAME, "Player died", onScreen: true);
    }

    [RuntimeDebug] private void Heal() => m_health = 100;   // button in the Runtime tab

    [ConsoleCommand("god")]                                  // typed as: god true
    private static void GodMode(bool enabled = true) => Logger.LogOnScreen(EDebugCategory.GAME, "God " + enabled);
}
```

EDITOR TOOLING

- **Window/Dhs5 Utility/Debugger**: **Categories**, **Runtime** (play mode; search objects by name or members with `m:`; category filter) and **Settings**.
- **Window/Dhs5 Utility/Console** ("D5 Console"): stored logs, Clear, per-category level filters and a command line. EDITOR and BOTH commands work outside play mode.

GOOD TO KNOW

- A lower level means more important. A log or warning prints if its category is active and the message level ≤ the category level. Errors and asserts always print.
- The on-screen console, `LogOnScreen` and runtime inspection only exist in the editor and development builds (`DEBUG`).
- Command arguments are split on spaces, so strings can't contain spaces. Vectors and colours are written `1,2,3`. Supported types: bool, int, float, string, enum, vectors, Color.
- The built-in command `timescale FLOAT` is available out of the box.

04 Updater

`Dhs5.Utility.Updates`

A central update system that inserts its own stages into Unity's PlayerLoop. It provides configurable **update channels** (stage, order, condition, frequency, timescale, realtime), **delayed calls** (frames, seconds, realtime, predicate), **timelines** (fixed-duration updates with events) and per-frame time values: `Updater.Time`, `DeltaTime`, `RealTime`, `Frame`, `TimePaused`.

KEY TYPES

`Updater (static)`	`RegisterChannelCallback(bool, EUpdateChannel, UpdateCallback)`, `SetChannelEnable / Timescale / Frequency`, `PauseTime(bool)`. Per-stage events `OnUpdateClassic`, … and one-shots `OneShotClassicUpdate`, …
`EUpdateChannel`	Enum generated from the channel assets (`UC_<NAME>`). `CLASSIC` always exists.
`EUpdatePass`	Stages: `AFTER_EARLY_UPDATE`, `CLASSIC_UPDATE` (just before Update), `AFTER_UPDATE`, `AFTER_LATE_UPDATE`, `BEFORE_FIXED_UPDATE`, `AFTER_PHYSICS_FIXED_UPDATE`.
`EUpdateCondition`	Generated enum. Each value maps to an `UpdateConditionObject` (`bool CanUpdate()`). `ALWAYS` is built in.
`Delayed calls`	`CallInXFrames`, `CallInXSeconds`, `CallInXRealtimeSeconds`, `CallWhenTrue / False`, each with `out DelayedCallHandle` (`Kill()`, `GetTimeLeft()`) and optional pass and condition.
`Timelines`	`CreateTimelineInstance(channel, duration, out handle, loop, timescale, events)` or from an `IUpdateTimeline` / `ScriptedUpdateTimeline`. The handle has `Play`, `Pause`, `SetTime`, `Restart`, `Kill` and the events `Updated`, `Ended`, `CustomEventTriggered`.

SETUP

1. **Window > Dhs5 Utility > Updater → Settings → Create new asset**. This creates `Resources/Updater/Updater.asset`.
2. **Channels** tab: **ADD NEW CHANNEL**, set the pass, order, condition, frequency, timescale and realtime, then press **UPDATE CHANNEL SCRIPT**.
3. **Conditions** tab: **ADD NEW CONDITION**, assign an `UpdateConditionObject` asset, then press **UPDATE CONDITION SCRIPT**. For a custom condition, subclass `UpdateConditionObject` and add `[CreateAssetMenu(menuName = UpdateConditionObject.MenuPath + "…")]`.

EXAMPLE

```
private DelayedCallHandle m_delay;
private UpdateTimelineInstanceHandle m_pulse;

private void OnEnable()
{
    Updater.RegisterChannelCallback(true, EUpdateChannel.CLASSIC, OnTick);
    Updater.CallInXSeconds(2f, () => Debug.Log("2 seconds later"), out m_delay);

    if (Updater.CreateTimelineInstance(EUpdateChannel.CLASSIC, 1.5f, out m_pulse, loop: true))
    {
        m_pulse.Updated += dt => transform.localScale = Vector3.one * (1f + m_pulse.NormalizedTime);
        m_pulse.Play();                       // timelines are created stopped
    }
}
private void OnDisable()
{
    Updater.RegisterChannelCallback(false, EUpdateChannel.CLASSIC, OnTick);
    m_delay.Kill(); m_pulse.Kill();
}
private void OnTick(float deltaTime) => transform.Rotate(0f, 90f * deltaTime, 0f);
```

GOOD TO KNOW

- Each non-`ALWAYS` condition needs an object assigned. Without one, the channel or delayed call never runs and a warning is logged.
- Fixed-update channels always receive `Time.fixedDeltaTime` and ignore frequency and timescale. With a custom frequency, the callback receives the time accumulated since its last call.
- A delayed call with a non-zero delay starts counting at the next frame. A zero delay fires this frame, or immediately if the stage has already run.
- At most 32 channels. A timeline's duration must be > 0.

05 Save & Load

`Dhs5.Utility.SaveLoad`

Saves are split into **categories**. During a save, game code fills one data object (`BaseSaveSubObject`) per category, plus an optional save info. These are serialized to JSON and written to disk atomically. Loading reads the file and passes each category's data to the registered `ILoadable` objects, in a **configurable load order**, as a coroutine.

KEY TYPES

`SaveManager (static)`	Save: `StartSaveProcess()`, `CreateCategoryData<T>(cat)`, `CreateSaveInfo<T>()`, `CompleteSaveProcess(param)`. Load: `StartLoadProcess(param)`, events `LoadCompleted` / `LoadCancelled`, `Register(bool, ILoadable, cat)`, `TryGetSubObject<T>`, `GetSaveVersion()`.
`ESaveCategory`	Enum generated from the categories in `Resources/Save/SaveAsset.asset`.
`BaseSaveSubObject`	Subclass it for each category's data (serializable fields). `BaseSaveInfo` is the base for save info, such as a slot header.
`ILoadable`	`LoadCoroutine(cat, iteration, subObject)`, `CanLoad(cat, iteration)`, `LoadDefault(cat)`.
`SaveProcessModifier`	Optional ScriptableObject that overrides save paths (slots), encryption, disk read/write, the serializer and error handling.
`ISaveParameter`	Gives the file extension and encoding for a save or load.

SETUP

1. **Window > Dhs5 Utility > Save & Load** → **Settings** → **Create new asset**.
2. **Categories** tab: **ADD NEW CATEGORY**, then press **UPDATE CATEGORY SCRIPT**.
3. **Load** tab: arrange the **LOAD ORDER**. Repeating a category loads it again, with `iteration` = 1, 2, ...
4. Optional: in Settings, **CREATE** a modifier and set the **Version**.

EXAMPLE

```
public class PlayerData : BaseSaveSubObject { public Vector3 position; public int gold; }

public class Player : MonoBehaviour, ILoadable
{
    public int gold;
    void OnEnable()  => SaveManager.Register(true, this, ESaveCategory.PLAYER); // your category
    void OnDisable() => SaveManager.Register(false, this, ESaveCategory.PLAYER);

    public void Save()
    {
        if (!SaveManager.StartSaveProcess()) return;
        var data = SaveManager.CreateCategoryData<PlayerData>(ESaveCategory.PLAYER);
        data.position = transform.position; data.gold = gold;
        SaveManager.CompleteSaveProcess();
    }
    public bool CanLoad(ESaveCategory cat, uint iteration) => iteration == 1;
    public IEnumerator LoadCoroutine(ESaveCategory cat, uint iteration, BaseSaveSubObject sub)
    {
        var data = (PlayerData)sub;          // copy the values, don't keep the object
        transform.position = data.position; gold = data.gold;
        yield break;
    }
    public void LoadDefault(ESaveCategory cat) => gold = 0;
}
// Load: SaveManager.LoadCompleted += OnLoaded; SaveManager.StartLoadProcess();
```

GOOD TO KNOW

- Without a modifier there is one file, `persistentDataPath/Save/SAVE` + extension (`.txt` by default). Writes go to a `.tmp` file first, so a failed write keeps the previous save.
- `UnityEngine.Object` references can't be saved; store IDs (for example database UIDs). Loaded objects are destroyed on the next save or load.
- Loading only works in play mode. An exception in a loadable cancels the load (`LoadCancelled`) unless the modifier handles it. `LoadDefault` is called for categories missing from the file.
- Renamed categories still load from old saves. Bump **Version** when the data format changes and branch on `GetSaveVersion()`.

06 UI

`Dhs5.Utility.UI`

Replacements for the uGUI controls, all built on `UISelectable`. They raise **plain C# events** and track hover, left/right press, selection and interactable state. Visual feedback comes from stackable **transitioners** driven by reusable **transition assets**. **Nav boxes** handle keyboard and gamepad navigation.

CONTROLS

`UIButton`	`Clicked`, `TrySimulateClick()`
`UIToggle`	`IsOn`, `ValueChanged`, `SetIsOnWithoutNotify`. Grouped with `UIToggleGroup`.
`UISlider / UIScrollbar`	`Value`, `ValueChanged`, `SetValueWithoutNotify`
`UIScrollRect`	Scroll view with normalized positions
`UIDropdown`	`AddOptions`, `ClearOptions`, `Show/Hide`, `MultiSelect` (bit-mask value)
`UIScrollList`	Carousel selector: `SetNext`, `AnimateNext`, `WrapAround`
`UIInputField`	TMP field: `Text`, `ValueChanged`, `Submitted`, `EditEnded`

TRANSITIONS & NAVIGATION

`UIGenericTransitioner`	Target graphics + a list of (asset, preset) instances
`Transition assets`	Active, Alpha, Color, Enable, Rect Anchor / Offset / Pivot / Size, Scale, Sprite, Text Size, Text Style. Each preset has a value and duration per state: Normal, Highlighted, Pressed, Selected, Disabled.
`UIStandaloneTransitioner`	Reacts to the pointer without a `UISelectable`
`UIListNavBox / UIGridNavBox`	1D list (`Axis`) or grid, `WrapAround`
`UISelectionTransferer`	Forwards selection to another object

SETUP

1. Under a Canvas, use **GameObject** > **UI (Canvas)** > UI Button (+ Text), UI Toggle (+ Text), UI Slider, UI Scrollbar, UI ScrollView/..., UI Dropdown, UI ScrollList or UI Input Field. Each creates the full hierarchy with a transitioner.
2. Create transition assets via **Create** > **Dhs5 Utility** > **UI** > **Transition Asset** > ... and add presets with **+**.
3. On the transitioner, add an instance, pick the asset and preset, and list the target graphics.
4. Navigation: put controls inside a `UIListNavBox` / `UIGridNavBox`. Enable *Setup On Start* on the top-level box only.

EXAMPLE

```
[SerializeField] UIButton m_back; [SerializeField] UIToggle m_music;
[SerializeField] UISlider m_volume; [SerializeField] UIDropdown m_quality;

void Start()
{
    m_back.Clicked += () => gameObject.SetActive(false);
    m_music.ValueChanged += isOn => Debug.Log($"Music: {isOn}");
    m_volume.ValueChanged += v => AudioListener.volume = v;

    m_quality.ClearOptions();
    m_quality.AddOptions("Low", "Medium", "High");
    m_quality.SetValueWithoutNotify(1);
    m_quality.ValueChanged += i => QualitySettings.SetQualityLevel(i);
}
```

EXTENDING

- **Control:** inherit `UISelectable` and override the `OnBefore…/OnAfter…` hooks; the pointer and select handlers are sealed.
- **Transition:** inherit `UIGenericTransitionAsset<T, UITransitionPreset<T>>` (`ApplyValue`, `ApplyValueInstant`, ...) or `UITransitioner` (`UpdateState`).
- **Items:** `UIDropdownItem` / `UIScrollListItem`. **Box:** `UINavBox`.

GOOD TO KNOW

- Events are code-only (no UnityEvents). Controls created from the menus have built-in navigation and transition set to *None*; navigation comes from nav boxes.
- A transitioner only runs if it is in the control's *Transitioners* list. Transitioners run by `Priority`, and tweens use unscaled time.

07 Scene Management

`Dhs5.Utility.Scenes`

`SceneReference` is a serializable field you drag a scene asset onto. It stores the scene's **name, path and build index**, so they are available in builds where `SceneAsset` doesn't exist. The **Scenes window** is a browser for every scene in Build Settings, grouped by folder, with one-click open, load, play and build toggles.

KEY TYPES

`SceneReference`	`SceneIndex`, `SceneName`, `ScenePath`, `IsLoaded`, `LoadScene(LoadSceneMode = Single)`, `UnloadScene(UnloadSceneOptions)`. Both methods return `bool`.
`SceneReferenceDrawer`	Scene asset field and a Refresh button. When expanded, shows the path and build index.
`ScenesWindow`	Editor window, opened from **Window/Dhs5 Utility/Scenes**.

USAGE

1. Declare `[SerializeField] SceneReference m_scene;` and drag a scene onto it.
2. Make sure the scene is in Build Settings; `LoadScene` uses the build index.
3. The cached data refreshes when the object is serialized in the editor, or with the drawer's Refresh button.

EXAMPLE

```
using Dhs5.Utility.Scenes;
using UnityEngine;
using UnityEngine.SceneManagement;

public class LevelFlow : MonoBehaviour
{
    [SerializeField] private SceneReference m_menuScene;
    [SerializeField] private SceneReference m_hudScene;

    public void EnterLevel()
    {
        if (!m_hudScene.IsLoaded) m_hudScene.LoadScene(LoadSceneMode.Additive);
    }

    public void LeaveLevel()
    {
        m_hudScene.UnloadScene();
        if (!m_menuScene.LoadScene())
            Debug.LogError($"{m_menuScene.SceneName} is not in Build Settings");
    }
}
```

SCENES WINDOW

- **Active Scenes** panel (collapsible, resizable): the scenes currently open.
- **Scene Browser**: searchable and grouped by parent folder. Each row has:
  - a build toggle (writes `EditorBuildSettings`), the name (click to ping the asset) and the build index;
  - **Add / Load / Unload** (additive), **Open / Remove** (single) and **Play** (opens the scene alone and enters Play Mode).
- Row colours: green = loaded, yellow = open but unloaded. Play and Open are disabled in Play Mode.

GOOD TO KNOW

- `LoadScene` is synchronous and returns false when the build index is -1 (scene not in Build Settings). `UnloadScene` is async and doesn't return the operation.
- If Build Settings are reordered or the scene is moved, the cached index and path can be stale until the next refresh or reserialization.
- The window only lists scenes in Build Settings. Two scenes with the same name will clash in the active-scene tracking.
- **Open** and **Play** don't ask to save modified scenes first.

08 Player Loop

`Dhs5.Utility.PlayerLoops`

A safe, shared way to modify Unity's low-level `PlayerLoop`. Modifiers register at startup and are applied together, **in priority order**, after the first scene loads. Individual systems can be disabled and re-enabled at runtime, and the original loop is **restored** on quit and at the next play session. The Updater is built on this system.

KEY TYPES

`IPlayerLoopModifier`

`int Priority` (higher numbers are applied later; the Updater uses 0) and `PlayerLoopSystem ModifyPlayerLoop(PlayerLoopSystem)`.

`PlayerLoopManager`

- `RegisterModifier(modifier)` and the `PlayerLoopInitialized` event.
- `DisableSystem(Type)`, `ReenableSystem(Type)`, `IsSystemEnabled(Type)`.
- `AddCustomMainSystemBefore/After/AtIndex` and `AddCustomSubSystemAtIndex/AtLast`.
- `ResetPlayerLoop()` restores the loop from before any changes; `ResetPlayerLoopToDefault()` applies Unity's default loop.

USAGE

1. Implement `IPlayerLoopModifier`. Identify each inserted system with an empty marker `struct` type so it can be found and toggled by `Type`.
2. Register it from `[RuntimeInitializeOnLoadMethod]` with `SubsystemRegistration` or `BeforeSceneLoad`.
3. At `AfterSceneLoad`, the manager chains every modifier over the current loop (systems added by other packages are kept), applies the result and fires `PlayerLoopInitialized`.

EXAMPLE

```
public class NetTickModifier : IPlayerLoopModifier
{
    public struct NetTick { }              // marker type for this system
    public int Priority => 10;             // after the Updater (0)

    [RuntimeInitializeOnLoadMethod(RuntimeInitializeLoadType.BeforeSceneLoad)]
    private static void Init() => PlayerLoopManager.RegisterModifier(new NetTickModifier());

    public PlayerLoopSystem ModifyPlayerLoop(PlayerLoopSystem loop)
    {
        for (int i = 0; i < loop.subSystemList.Length; i++)
            if (loop.subSystemList[i].type == typeof(PreLateUpdate))
            {
                var subs = new List<PlayerLoopSystem>(loop.subSystemList[i].subSystemList);
                subs.Insert(0, new PlayerLoopSystem { type = typeof(NetTick), updateDelegate = Tick });
                loop.subSystemList[i].subSystemList = subs.ToArray();
            }
        return loop;
    }
    private static void Tick() { /* ... */ }
}
// Later, at runtime:
PlayerLoopManager.DisableSystem(typeof(NetTickModifier.NetTick));
```

EDITOR TOOLING

- **Window/Dhs5 Utility/Player Loop**: a visualizer in edit mode and a manager in play mode.
  - Toolbar: search by type name, a CPU profiler toggle (per-system ms) and Refresh.
  - Custom systems are drawn in cyan. In play mode, each system has an enable toggle.

GOOD TO KNOW

- `RegisterModifier` returns false after `AfterSceneLoad`. After that, use the `AddCustom…` and `DisableSystem` methods instead.
- The Updater inserts at fixed indexes in Unity's default loop. A modifier with a priority below 0 that adds or removes main systems can break it.
- `DisableSystem` only searches main systems and their direct sub systems. `ResetPlayerLoopToDefault()` also removes systems added by third-party packages.

09 Ticket System

`Dhs5.Utility.TicketSystem`

Hands out lightweight unique IDs called **tickets**. The caller keeps the ticket and uses it later to remove or modify what it registered. Typical uses are cancellable modifiers, lock counters and handle-based registries, without exposing references or delegates. No setup or assets are needed; create a handler with `new`.

KEY TYPES

`Ticket`	Serializable struct wrapping a `ulong` UID. `GetUID()`. Its drawer shows the UID read-only.
`TicketHandler`	Issues tickets in sequence starting at 1. `virtual GetTicket()`, `virtual Reset()`.
`TicketHandlerList`	Remembers the tickets it issued: `GetTicket()`, `Remove(t)`, `Contains(t)`, `HasContent()`, `Count`.
`TicketHandlerDictionary<T>`	Maps tickets to values: • `Register(value)` → `Ticket`, `Unregister(t [, out value])`, `SetValue(t, v)` • `TryGetValue`, `GetValue` / `this[t]`, `Contains`, `HasContent()`, `Count` • Enumerable as `KeyValuePair<Ticket, T>`

EXAMPLE: STACKABLE SPEED MODIFIERS AND STUN LOCKS

```
using Dhs5.Utility.TicketSystem;
using UnityEngine;

public class SpeedModifiers : MonoBehaviour
{
    private readonly TicketHandlerDictionary<float> m_multipliers = new();
    private readonly TicketHandlerList m_stunLocks = new();

    public Ticket AddMultiplier(float mult) => m_multipliers.Register(mult);
    public bool RemoveMultiplier(Ticket ticket) => m_multipliers.Unregister(ticket);

    public Ticket Stun() => m_stunLocks.GetTicket();
    public void ReleaseStun(Ticket ticket) => m_stunLocks.Remove(ticket);

    public float GetSpeed(float baseSpeed)
    {
        if (m_stunLocks.HasContent()) return 0f;
        foreach (var kvp in m_multipliers) baseSpeed *= kvp.Value;
        return baseSpeed;
    }
}

// Caller side
Ticket slow = speed.AddMultiplier(0.5f);
...
speed.RemoveMultiplier(slow);
```

EXTENDING

- Subclass `TicketHandler` the way `TicketHandlerList` does: override `GetTicket()`, call `base.GetTicket()`, then store the result.

GOOD TO KNOW

- UIDs start at 1, so `default(Ticket)` (UID 0) works as "no ticket". `Reset()` restarts at 1, so old tickets can collide with new ones.
- `Ticket` doesn't define `==` or `Equals`. Compare tickets with `GetUID()`.
- `GetValue` and the indexer throw for unknown tickets; use `TryGetValue`. The indexer is read-only; use `SetValue` to change a value.
- Handlers are not thread-safe.

10 Attributes

`global namespace · editor drawers only`

Inspector attributes for common editor needs. Put them on serialized fields; they have no effect at runtime. Each one is a `PropertyAttribute` with a matching drawer in the same file, inside `#if UNITY_EDITOR`.

REFERENCE

`[ReadOnly]`	Always read-only.
`[ReadOnly(inEditor, atRuntime)]`	Writable in edit mode if `canWriteInEditor`, and in play mode if `canWriteAtRuntime`.
`[ReadOnly(param, inverse)]`	Read-only while the named serialized bool is true (or false with `inverse`).
`[Show(param, inverse)]`	Shown only while the named bool is true (or false with `inverse`).
`[HelpBox(text, EType)]`	Help box above the field. `EType`: `NONE`, `INFO`, `WARNING`, `ERROR`.
`[Tag]`	`string` → tag dropdown.
`[Layer]`	`int` → layer dropdown.
`[VectorRange(min, max)]`	`Vector2` / `Vector2Int` → min-max slider (x = min, y = max).
`[FolderPicker(root?)]`	On a `string` (stores "Assets/...") or an `Object` field: folder dialog button. Right-click: copy path, ping folder.
`[Creator]`	On a ScriptableObject field: **Create** button that saves a new asset and assigns it. For an abstract type it lists the concrete subtypes.
`[FoldoutContent]`	On a ScriptableObject field: edit the referenced asset's fields inline in a foldout.
`[DateReadOnly]`	On a `SerializableDate`: a formatted read-only label (see Custom Properties).

EXAMPLE

```
public class EnemySpawner : MonoBehaviour
{
    [SerializeField, Tag] private string m_targetTag;
    [SerializeField, Layer] private int m_projectileLayer;
    [SerializeField, VectorRange(0f, 10f)] private Vector2 m_spawnDelay;
    [SerializeField, VectorRange(1, 20)] private Vector2Int m_waveSize;

    [HelpBox("Enables the patrol settings below", HelpBoxAttribute.EType.INFO)]
    [SerializeField] private bool m_patrols;
    [SerializeField, Show(nameof(m_patrols))] private float m_patrolSpeed;

    [SerializeField, ReadOnly(true)] private int m_maxEnemies = 10;  // editable in edit mode only
    [SerializeField, ReadOnly] private int m_aliveCount;             // always read-only

    [SerializeField, FolderPicker("Assets")] private string m_lootFolder;
    [SerializeField, Creator] private SpawnTable m_table;            // SpawnTable : ScriptableObject
    [SerializeField, FoldoutContent] private SpawnTable m_bossTable;
}
```

EXTENDING

- Same pattern as the existing attributes: a `PropertyAttribute` subclass plus a `[CustomPropertyDrawer]` inside `#if UNITY_EDITOR`.

GOOD TO KNOW

- `Show` and `ReadOnly(param)` look up the bool from the root of the object, so they don't work inside nested classes or structs. An unknown param means "always shown".
- `Show` draws a single line, so arrays, lists and structs are clipped.
- Unity uses one drawer per field, so don't combine drawer attributes (e.g. `[Show]` with `[ReadOnly]`).
- `ReadOnly` is ambiguous in files with `using Unity.Collections;`.

11 Custom Properties

global namespace

Serializable wrapper types, most with their own drawers, for data Unity can't serialize well on its own: per-enum tables, optional values, dates and `System.Type` references. Declare them as serialized fields; no setup is needed.

REFERENCE

`EnabledValue<T>`	An optional value with a toggle. `Value`, `IsEnabled(out T)`, `SetValue(enabled, value)`. The drawer shows the value only when the toggle is on.
`EnumValues<TEnum, U>`	One `U` per enum value. Indexer `this[TEnum]` (get and set) and a `KeyValuePair` enumerator. The drawer shows a row per enum name.
`EnumFlagValues<TE, TF, U>`	Like EnumValues, but the flags field `TF` (`Flag`) activates the entries (bit `1 << (int)te`). `TryGet(te, out U)` returns false for inactive entries; the enumerator yields active entries only.
`SerializableDate`	Date struct with the same fields as `DateTime`, built from a `DateTime` or from day/month/year...; `Now`. `ToStringDate(us)` gives dd/MM/yyyy (MM/dd/yyyy if `us`); `ToFullStringNoSeparator(…)` gives yyyyMMddHHmm[ss][fff]. Show it with `[DateReadOnly]`.
`SerializableMonoScript(<T>)`	Drag a script asset; read `Type` at runtime. The generic version rejects scripts not assignable to `T`.
`SerializableType<T>`	Stores any `System.Type` (generic types included) as a string. Implicit conversions to and from `Type`. No drawer; assign it from code.

EXAMPLE

```
public enum EDifficulty { EASY, NORMAL, HARD }
[Flags] public enum EDifficultyMask { EASY = 1 << 0, NORMAL = 1 << 1, HARD = 1 << 2 }

public class DifficultyConfig : ScriptableObject
{
    [SerializeField] private EnumValues<EDifficulty, float> m_enemyHealthMult;
    [SerializeField] private EnumFlagValues<EDifficulty, EDifficultyMask, int> m_bonusLives;
    [SerializeField] private EnabledValue<float> m_timeLimit;
    [SerializeField] private SerializableMonoScript<MonoBehaviour> m_aiBrain;
    [SerializeField, DateReadOnly] private SerializableDate m_lastEdited;

    public float HealthMult(EDifficulty d) => m_enemyHealthMult[d];
    public int BonusLives(EDifficulty d) => m_bonusLives.TryGet(d, out var lives) ? lives : 0;
    public bool HasTimeLimit(out float seconds) => m_timeLimit.IsEnabled(out seconds);
    public Type AIBrainType => m_aiBrain.Type;

    private void OnValidate() => m_lastEdited = SerializableDate.Now;
}
```

GOOD TO KNOW

- The enum tables are indexed by the enum's int value. Use non-negative, dense enums. For `EnumFlagValues`, flag bit *i* must match enum value *i*.
- The tables are only sized when the inspector draws them. Before that, reads return `default` and log "Enum Values ... not complete".
- `EnumValues.GetSimpleEnumerator()` returns an `IEnumerator`, so it can't go directly in a `foreach`. The `EnumFlagValues` version returns an `IEnumerable`.
- The drawers look up the private field names (`m_value`, `m_enumValues`, ...). Renaming them breaks the drawers and existing data.

12 General Utility

`global` · `Dhs5.Utility` · `.GUIs` · `.Editors`

The shared toolbox the other systems are built on: **extension methods** for vectors, collections, enums and components, small **runtime helpers**, and **editor-only** helpers for creating and managing assets, drawing IMGUI, reflection on serialized properties and **generating C# scripts**. Several systems depend on it (e.g. `IsValid()`), so it must ship with them.

RUNTIME

`ExtensionMethods` — Global namespace, always available.

- **Vectors:** `ToVector2/3` (+ `XZ`, `YZ`), `SetX/Y/Z`, `Min/Max`, `IsApproximatelyZero`. A `Vector2` used as a range: `Contains`, `Clamp`, `GetRandomInRange`.
- **Collections:** `IsValid()` (not null and not empty), `IsIndexValid(i [, out v])`, `Swap`, `Copy`, `Sum`, `Average`, `InitWithConstantValue`, `InitWithRange`, `ToString(separator, showIndex)`.
- **Random:** `GetRandom()`, `TryGetRandomWithPredicate`, `GetRandomWithExceptions`.
- **Misc:** `GetFlags<T>()`, `GetFlagsIndex()`, `Color.SetAlpha`, `LayerMask.Contains(layer)`, `TryParseFloat` (accepts "," or "."), `CopyToClipboard`, `TryGetComponentInParent/InChildren`, `bool.ToInt()` / `int.ToBool()`.

`UtilityMethods` — `FlatDistance(v1, v2)` (XZ plane), `EnsureDirectoryExistence`, `EnsureAssetParentDirectoryExistence`.

`GUIHelper` — IMGUI (runtime and editor): preset colours and styles, `DrawRect`, and `GUIColorScope` / `GUIContentColorScope` / `GUIBackgroundColorScope`.

EDITOR ONLY

`EditorDataUtility` — `CreateScriptableAsset<T>(path)`, `CreatePrefabWithComponent`, `CreateOrOverwriteTextAsset`, `Duplicate`, `DeleteAsset`, `RenameAsset`, `MoveAssetToFolder`, sub assets (`AddAssetToOtherAsset`, `GetSubAssets`, `EnsureAssetValidity`), `GetAllChildTypes(type)`.

`EditorUtils` — `FullPingObject`, `SelectAndFocusAsset`, `TriggerAssetRename`, copy / cut / paste asset, `DelayCall(seconds, cb)`, `OpenFolder`. Adds **Assets > Add to build settings** for scenes.

`EditorGUIHelper` — ~45 built-in icons (`RefreshIcon`, `AddIcon` …), `FolderPicker`, `ObjectList`, `ProgressBar`, `DrawChildTypeSelector`, `LabelWidthScope`.

`SerializedPropertyExtensions` — `property.GetValue<T>()` / `SetValue<T>(v)` via reflection. `ReflectionUtility`: `GetFieldOrPropertyValue`, `SetFieldOrPropertyValue`, `GetBaseClassesAndInterfaces`.

`ScriptWriter / EnumWriter` — Generate C# source: `AppendUsing`, `AppendField`, `AppendMethod`, `ToString()`. `EnumWriter` is what produces the generated enums of the Debugger, Updater and Save systems.

EXAMPLE

```
public class LootDropper : MonoBehaviour
{
    [SerializeField] private List<GameObject> m_loot;
    [SerializeField] private Vector2Int m_count = new(1, 4);    // used as a min/max range

    public void Drop()
    {
        if (!m_loot.IsValid()) return;                    // null or empty
        int count = m_count.GetRandomInRange(maxInclusive: true);
        for (int i = 0; i < count; i++)
            Instantiate(m_loot.GetRandom(), transform.position.SetY(0f), Quaternion.identity);
    }
}

#if UNITY_EDITOR  // using Dhs5.Utility.Editors;
var data = EditorDataUtility.CreateScriptableAsset<LootTable>("Assets/Data/LootTable", triggerRename: true);
EditorUtils.FullPingObject(data);
var writer = new EnumWriter("Game", ScriptWriter.EProtection.PUBLIC, "ELoot",
    new[] { "COIN", "GEM" }, EnumWriter.EEnumType.BYTE);
EditorDataUtility.CreateOrOverwriteTextAsset("Assets/Scripts/ELoot.cs", writer.ToString());
#endif
```

GOOD TO KNOW

- `IsValid()` means *not null* ***and*** *not empty*; an empty list is "invalid".
- `CreateScriptableAsset` adds `.asset` if missing, creates missing folders and makes the path unique (it never overwrites).
- Editor helpers are compiled under `#if UNITY_EDITOR`; guard any call to them the same way in runtime scripts.
- `LifecycleManagementCompatibility.cs` stubs Unity 6.6's statics-cleanup attributes so the project still compiles on older versions.